॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Extended Course Essesntials

Today's Class (2/11/2026)

The problem with manually creating nodes

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider:

```
struct node n1, n2, n3;
```





What if the user enters:

How many nodes do you want?

1000

We cannot keep writing:

```
struct node n1, n2, n3, ...;
```



We need a way to request **memory** from the computer while the program is running.



This is called

Dynamic Memory Allocation.

Where Does a C Program Store Its Data?

Before learning `malloc()`, let's understand where our variables live

When a C program runs, it uses different memory regions for different purposes.

Code Segment

Data Segment

Heap

Stack

Four important regions

Each region has a different responsibility. Let's understand them one by one!

Code Segment: Where Instructions Live

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

A C program contains instructions that the computer must execute.
For example:

```
int main(void)
{
    int a = 10;
    int b = 20;
    printf("%d", a + b);
}
```

After compilation, the executable instructions are stored in the program's code region.

Code Segment

Executable machine instructions

The code segment stores instructions, not the ordinary values of your local variables.

Data Segment: Global and Static Variables

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider:

```
int total = 100;          // Global variable
int count;                // Global variable
int main(void)
{
    static int visits = 5;
    return 0;
}
```

These variables have **static storage duration**.

Initialized data region

Zero-initialized data region

- Initialized global/static variables are typically stored in the initialized data area.
- Uninitialized or zero-initialized global/static variables are typically stored in the Zero- initialized data area.

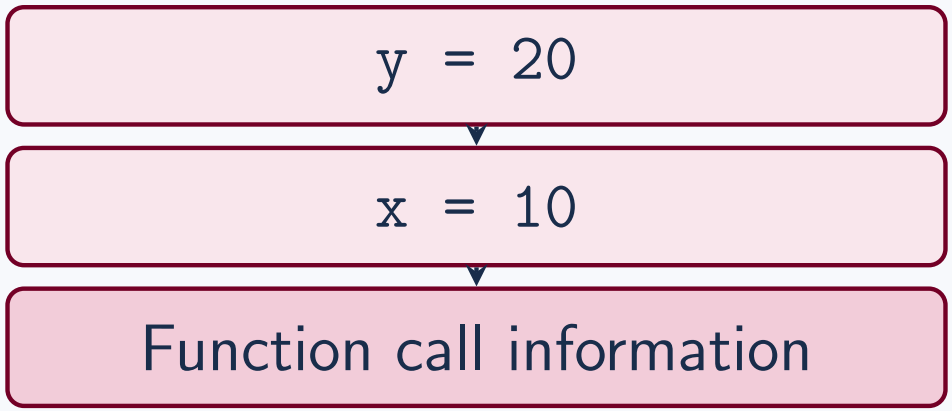
Stack: Function Calls and Local Variables

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider this function:

```
void calculate(void)
{
    int x = 10;
    int y = 20;
}
```

When `calculate()` is called, the program needs space for its function-call information and typically its local variables.



- Each function call typically gets its own stack frame.
- When the function returns, its stack frame is removed.

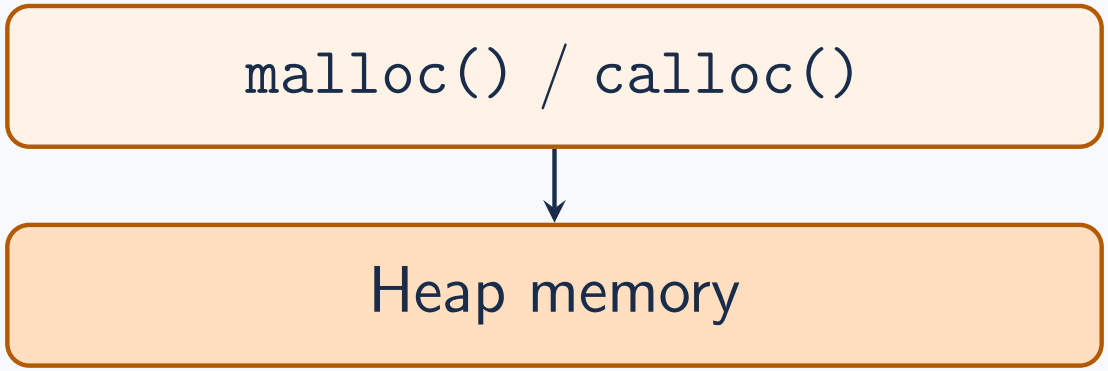
Heap: Dynamically Requested Memory

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

What if we don't know in advance how many elements we need?
For example:

```
int n;
printf("How many numbers? ");
scanf("%d", &n);
```

We cannot always decide the required storage at compile time.
This is where the **heap** becomes useful!



- Memory is requested dynamically while the program runs.
- The allocated memory remains available until released.
- The programmer must release it using `free()`.

Stack vs Heap: A Simple Example

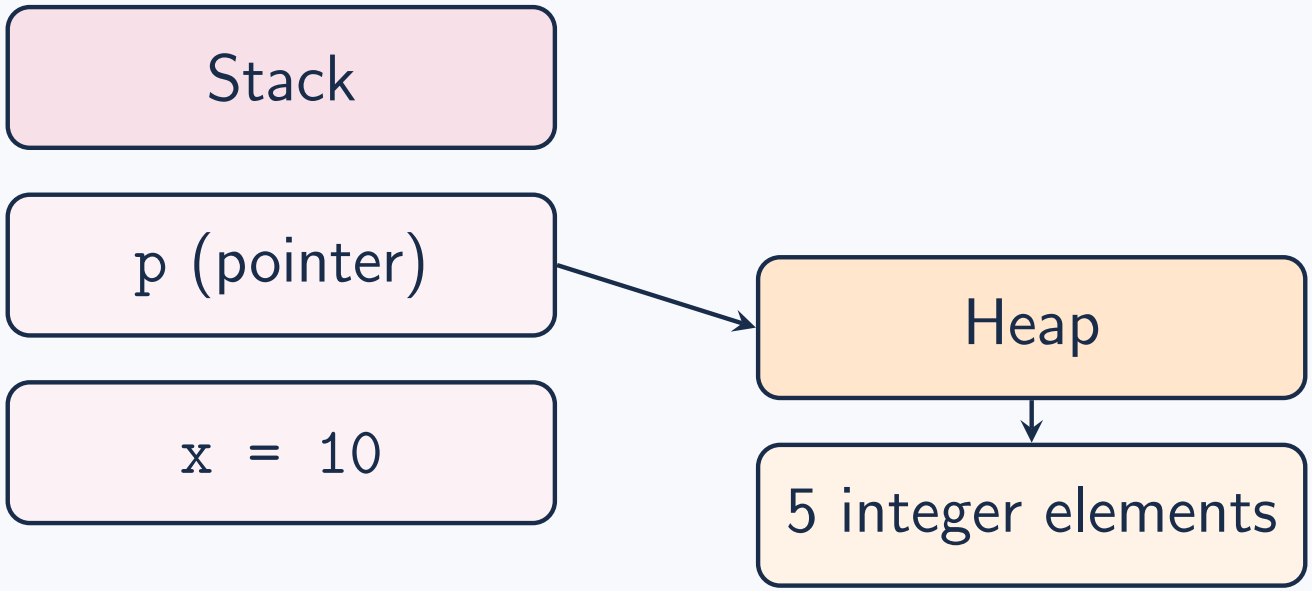
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider these two declarations:

```
int x = 10;

int *p = malloc(5 * sizeof(int));
```

Conceptually, assuming these declarations are inside a function:



- The pointer variable `p` is typically on the stack.
- The memory requested by `malloc()` comes from the heap.
- The pointer stores the address of the allocated memory.

How Long Does Each Memory Area Exist?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Let's compare the three important categories.

Memory	**Allocation**	**Lifetime**
Global / Static	Before or during program initialization	Entire program execution
Stack (typical)	Usually automatic	Until the relevant block or function execution ends
Heap	Dynamically requested	Until explicitly released using `free()`

A very important difference

Stack memory is generally managed automatically.
Heap memory allocated using `malloc()` or `calloc()` must be released explicitly using `free()`.

A Common Misconception: Pointer vs Memory

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Suppose we write:

```
int *p = malloc(5 * sizeof(int));
```

Are `p` and the allocated memory the same thing?



- `p` is a pointer variable.
- `malloc()` allocates a separate block of memory.
- `p` stores the address of that block.

Remember

A pointer is not the memory itself. It stores an address that can be used to access memory.

Quick Challenge: Where Does It Live?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Consider:

```
int global = 10;
void fun(void)
{
    int local = 20;
    int *p = malloc(sizeof(int));
    *p = 30;
    free(p);
}
```

Identify where each item is typically stored.

1. `global`
2. `local`
3. Pointer variable `p`
4. Memory allocated by `malloc()`

Think: Which of these requires an explicit `free()`?

Challenge Answer

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Item	**Typical location**
`global`	Data segment
`local`	Stack
Pointer variable `p`	Stack
Allocated integer	Heap

Important

The allocated memory must be released using `free(p)`.
The pointer variable itself is an ordinary local variable and is managed automatically when the function returns.

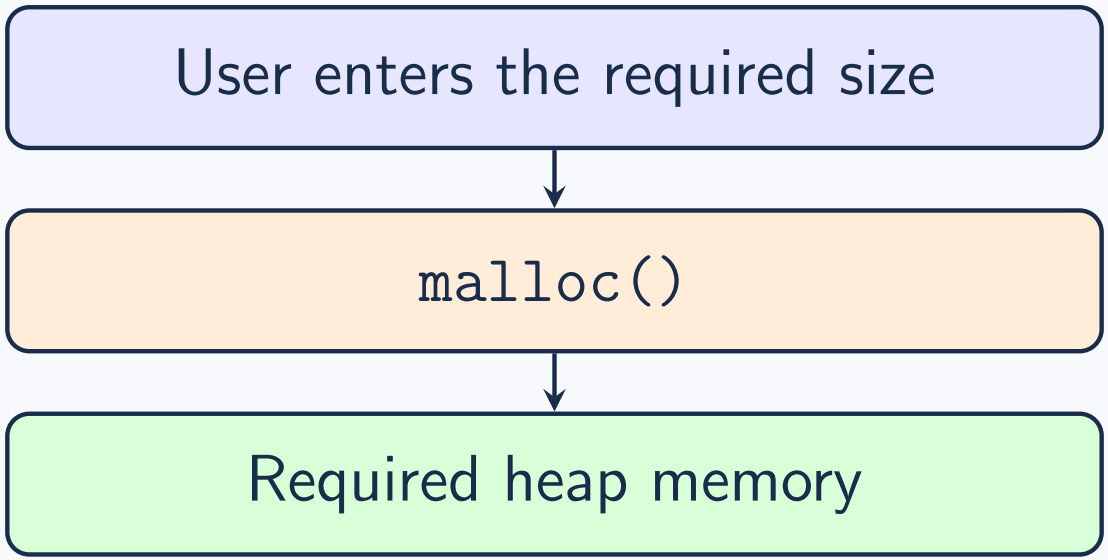
Why Do We Need Dynamic Memory Allocation?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Imagine asking the user:

How many numbers do you want to store?

The answer might be 5, 100, or even 10,000!
We need a way to request the required amount of memory while the program is running.



Next

Let's learn how C requests memory dynamically using `malloc()`!

Introducing `malloc()`: Requesting Memory at Runtime



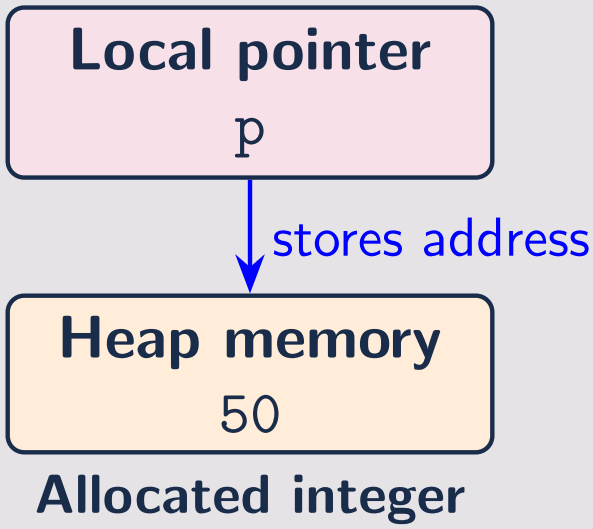
`malloc()` stands for **memory allocation**. It requests a specified number of bytes from the heap and returns the starting address of the allocated memory.

```
void *malloc(size_t numBytes);
```

Example: Allocate memory for one integer

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{   int *p;
    p = malloc(sizeof(int));
    if (p == NULL)
        return 1;
    *p = 50;
    printf("%d\n", *p);
    free(p);
    return 0;}
```

What happens in memory?



Creating a Linked-List Node Dynamically

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

Recall the structure of a linked-list node:

Structure definition

```
struct node {
    int val;
    struct node *ptr;
};
```

Create a node dynamically

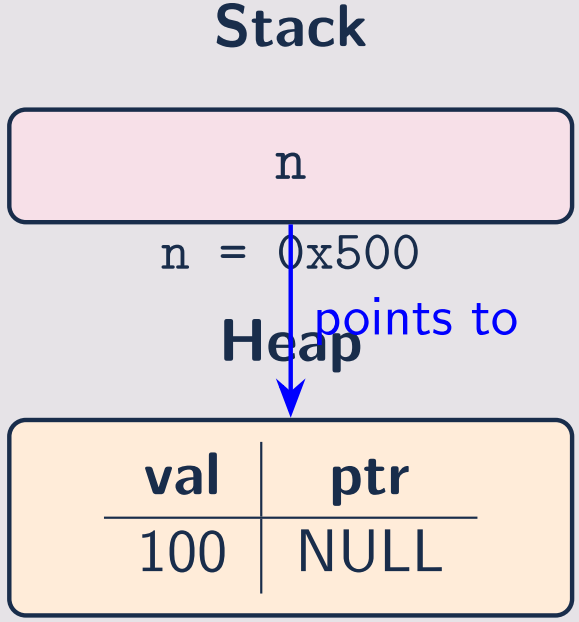
C code

```
struct node *n;

n = malloc(sizeof(struct node));

if (n != NULL) {
    n->val = 100;
    n->ptr = NULL;
}
```

What happens in memory?



Illustrative address: 0x500

Creating a Linked-List Node Dynamically

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Understand each statement

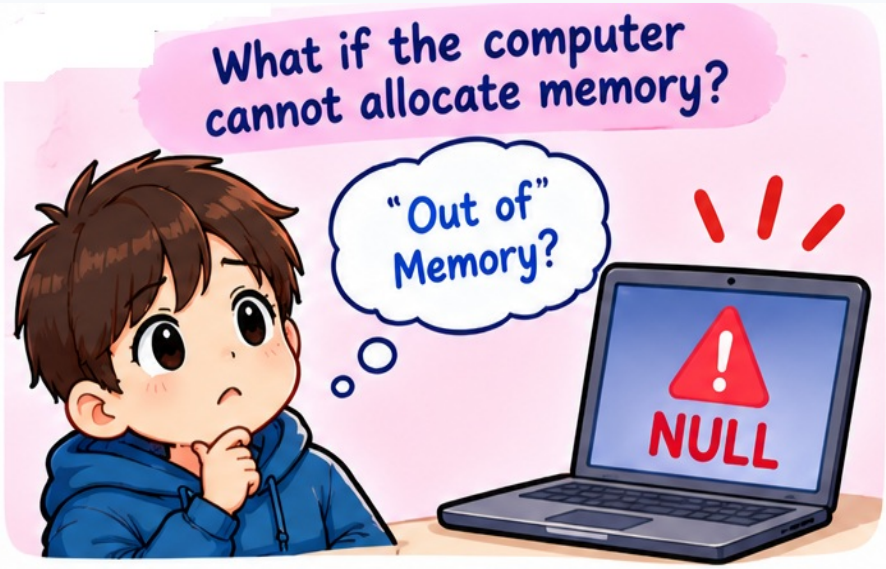
- `sizeof(struct node)` calculates the bytes needed for one node.
- `malloc()` requests that many bytes from the heap.
- The returned address is stored in pointer `n`.
- `n->val = 100;` stores 100 in the node's `val` field.
- `n->ptr = NULL;` indicates that this node does not point to another node yet.

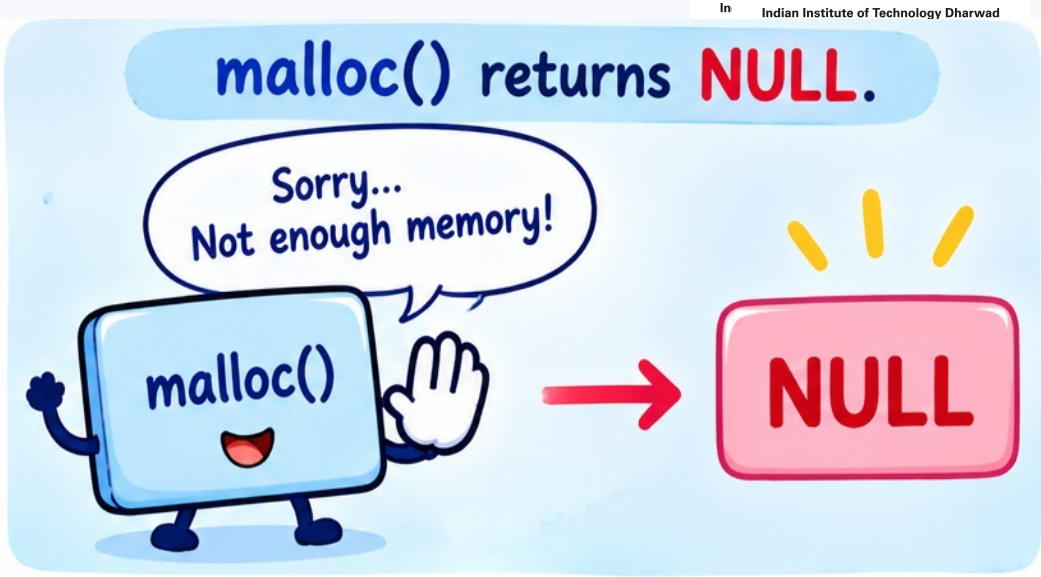
The important difference

`struct node *n;` declares a pointer variable.
`malloc(sizeof(struct node))` allocates memory for an actual node.

Always check whether malloc succeeded!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad





Therefore, always check:

```
struct node *n;

n = malloc(sizeof(struct node));

if (n == NULL) {
    printf("Memory allocation failed!\n");
    return 1;
}

n->val = 100;
n->ptr = NULL;
```

Check before using!

Remember:

Never dereference a pointer returned by `malloc()` before checking whether it is NULL.





॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Predict the Output

Consider this C code

```
int *p = malloc(sizeof(int));
*p = 50;
printf("%d\n", *p);
free(p);
```

What will be printed?

Option A

0

Option B

50

Option C

Garbage value

Option D

Compilation error

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Correct Answer: 50

Step 1: Allocate memory

```
int *p = malloc(sizeof(int));
```

A memory block large enough to store one integer is allocated. Its starting address is stored in `p`.

Step 2: Store the value

```
*p = 50;
```

The value `50` is stored in the allocated memory.

Step 3: Print the value

```
printf("%d\n", *p);
```

The expression `*p` accesses the stored integer. Therefore, the output is:

```
50
```

Lets creat a Linked-list Dynamically

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Now we have all the ingredients:

- A structure representing a node.
- A pointer to another node.
- malloc() to create nodes.
- NULL to indicate the end of the list.



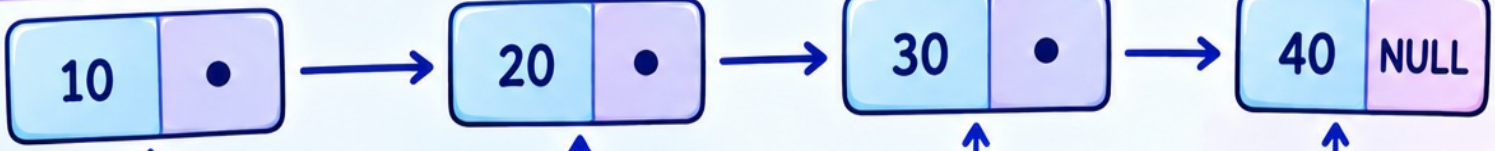
Suppose the user wants to store:

10 → 20 → 30 → 40 → NULL

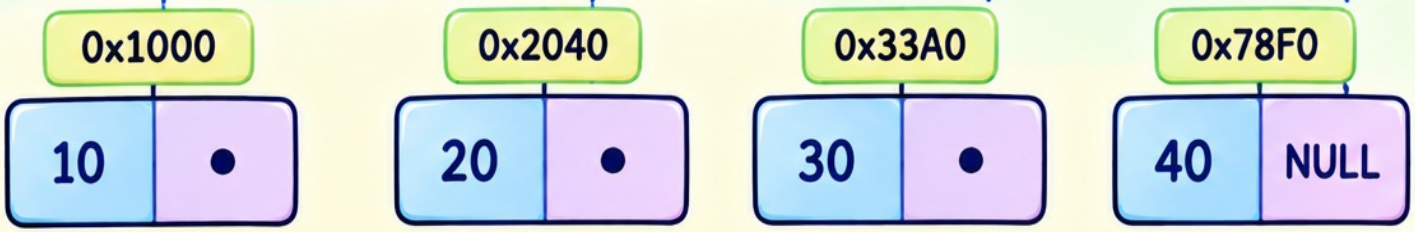
We can create every node separately.

malloc() malloc() malloc() malloc()

Conceptually:



Memory Locations (may be different!)



Notice:

These nodes may occupy completely **different** locations in memory.



What is the Head pointer?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad



We need some way to find the first node.

We introduce a special pointer called **head**.

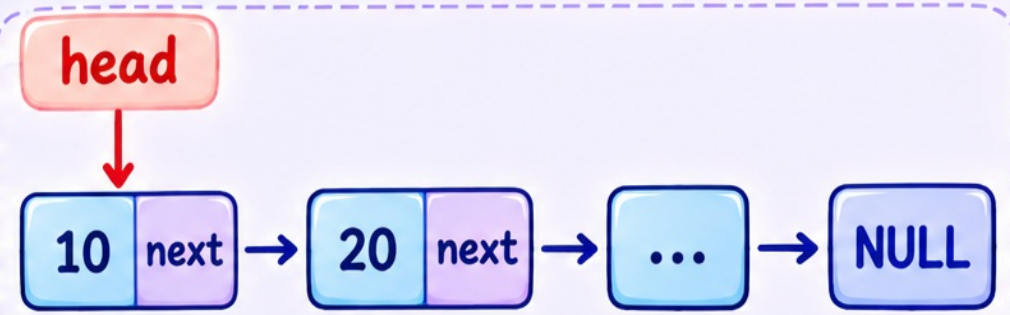
```
struct node *head = NULL;
```

The **head** pointer stores the address of the first node.

Why do we need it?



Imagine we have a chain of 100 nodes. If we lose the address of the first node, we may lose our ability to reach the entire list!



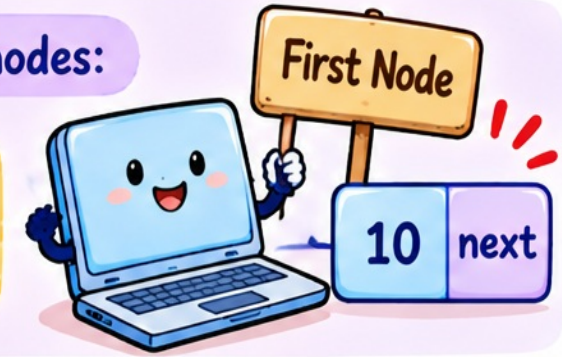
If the list is empty:

```
head = NULL;
```



If the list contains nodes:

head points to the first node.



A Complete C Program: Creating Two Nodes

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int val;
    struct node *ptr;
};
int main(void) {



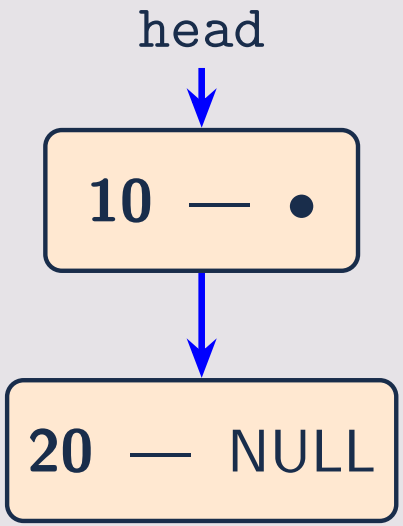






return 0;}
```

What does the program create?



Two connected nodes

Output

```
10
20
```

A Complete C Program: Creating Two Nodes

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int val;
    struct node *ptr;
};
int main(void) {
    struct node *head, *newNode;



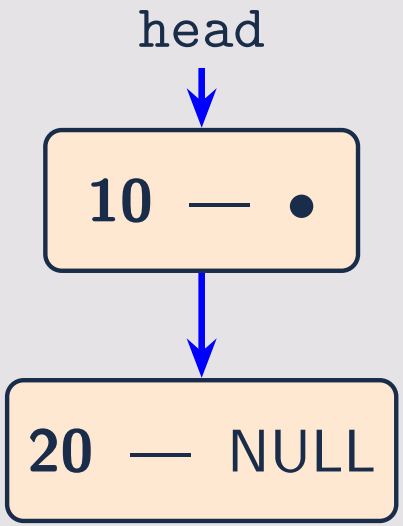






    return 0;}
```

What does the program create?



Two connected nodes

Output

```
10
20
```

A Complete C Program: Creating Two Nodes

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int val;
    struct node *ptr;
};
int main(void) {
    struct node *head, *newNode;
    head = malloc(sizeof(struct node));
    if (head == NULL)
        return 1;
    head->val = 10;
    head->ptr = NULL;



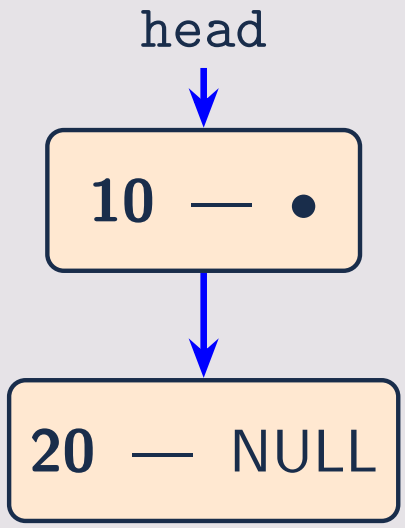






    return 0;}
```

What does the program create?



Two connected nodes

Output

```
10
20
```

A Complete C Program: Creating Two Nodes



```c
#include <stdio.h>
#include <stdlib.h>
struct node {
    int val;
    struct node *ptr;
};
int main(void) {
    struct node *head, *newNode;
    head = malloc(sizeof(struct node));
    if (head == NULL)
        return 1;
    head->val = 10;
    head->ptr = NULL;
    newNode = malloc(sizeof(struct node));
    if (newNode == NULL) {
        free(head);
        return 1;
    }
    newNode->val = 20;
    newNode->ptr = NULL;



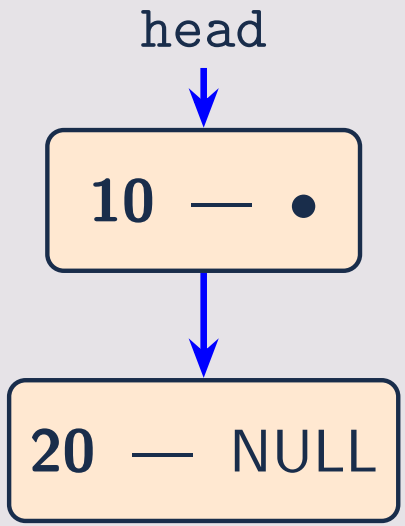






    return 0;}
```

What does the program create?



Two connected nodes

Output

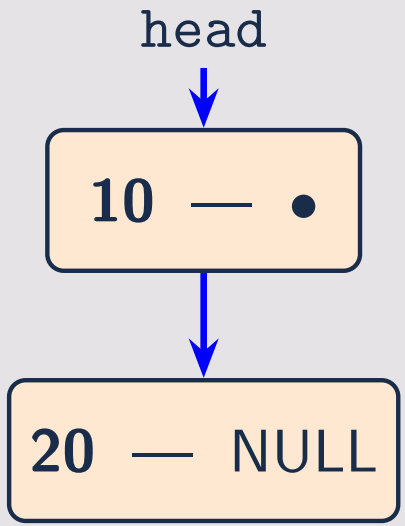
```
10
20
```

A Complete C Program: Creating Two Nodes



```c
#include <stdio.h>
#include <stdlib.h>
struct node {
    int val;
    struct node *ptr;
};
int main(void) {
    struct node *head, *newNode;
    head = malloc(sizeof(struct node));
    if (head == NULL)
        return 1;
    head->val = 10;
    head->ptr = NULL;
    newNode = malloc(sizeof(struct node));
    if (newNode == NULL) {
        free(head);
        return 1;
    }
    newNode->val = 20;
    newNode->ptr = NULL;
    head->ptr = newNode;
    printf("%d\n", head->val);
    printf("%d\n", head->ptr->val);
    free(newNode);
    free(head);
    return 0;}
```

What does the program create?



Two connected nodes

Output

```
10
20
```

Today's Class (4/11/2026)

Traversal function


```
void printChain(struct node *first) {
    struct node *temp;

    temp = first;

    while (temp != NULL) {
        printf("%d\n", temp->val);
        temp = temp->ptr;
    }
}
```

Complete function: Insert at beginning

```
struct node* insertBeginning(struct node *head, int x) {

    struct node *newNode;

    newNode = malloc(sizeof(struct node));

    if (newNode == NULL)
        return head;

    newNode->val = x;

    newNode->ptr = head;

    head = newNode;

    return head;
}
```

Complete function: Insert at the end




```
struct node* insertEnd(struct node *head, int x) {
    struct node *newNode, *temp;
    newNode = malloc(sizeof(struct node));
    if (newNode == NULL)
        return head;
    newNode->val = x;
    newNode->ptr = NULL;
    if (head == NULL)
        return newNode;
    temp = head;
    while (temp->ptr != NULL) {
        temp = temp->ptr;
    }
    temp->ptr = newNode;
    return head;
}
```

Complete function: Insert at a position

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
struct node* insertAtPosition( struct node *head, int x, int pos)
{
    struct node *newNode, *temp;
    if (pos < 1)
        return head;
    if (pos == 1)
        return insertBeginning(head, x);
    temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->ptr;
    }
    if (temp == NULL)
        return head;
    newNode = malloc(sizeof(struct node));
    if (newNode == NULL)
        return head;
    newNode->val = x;
    newNode->ptr = temp->ptr;
    temp->ptr = newNode;
    return head;
}
```

Complete function: Delete first node

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

```
struct node* deleteBeginning(struct node *head) {

    struct node *temp;

    if (head == NULL)
        return NULL;

    temp = head;
    head = head->ptr;

    free(temp);

    return head;
}
```

Complete function: Delete last node

```
struct node* deleteEnd(struct node *head) {

    struct node *temp;

    if (head == NULL)
        return NULL;

    if (head->ptr == NULL) {
        free(head);
        return NULL;
    }

    temp = head;

    while (temp->ptr->ptr != NULL) {
        temp = temp->ptr;
    }

    free(temp->ptr);

    temp->ptr = NULL;

    return head;
}
```

Delete at a position

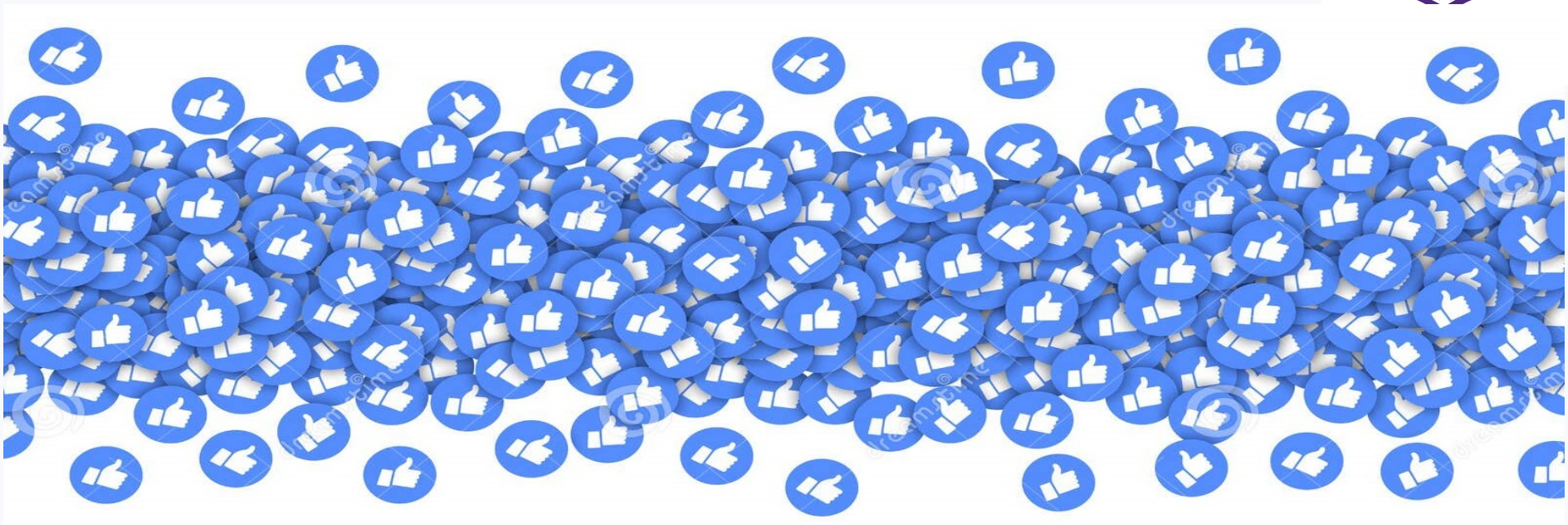
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

```
struct node* deleteAtPosition(
    struct node *head, int pos)
{
    struct node *temp, *del;
    if (head == NULL || pos < 1)
        return head;
    if (pos == 1)
        return deleteBeginning(head);
    temp = head;

    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->ptr;
    }

    if (temp == NULL || temp->ptr == NULL)
        return head;

    del = temp->ptr;
    temp->ptr = del->ptr;
    free(del);
    return head;
}
```



Thank you for your kind attention!!